

Actividad #3 — Juego de Memoria Pokémon



Objetivo

El propósito de esta actividad es que el estudiante diseñe y desarrolle, en lenguaje Java, un juego de memoria para dos jugadores, aplicando de manera rigurosa los principios fundamentales de la Programación Orientada a Objetos (POO). El proyecto debe evidenciar el uso correcto de clases abstractas, interfaces, herencia, polimorfismo, manejo estructurado de excepciones (try-catch) y el desarrollo de una interfaz gráfica de usuario (GUI) funcional e intuitiva.

Más allá de producir un juego operativo, se espera que el equipo demuestre capacidad de análisis y diseño de software: identificar correctamente las responsabilidades de cada clase, justificar las decisiones de arquitectura y aplicar buenas prácticas de programación (nombres descriptivos, separación de responsabilidades y organización del código en paquetes).

Descripción General del Sistema

El sistema consiste en un juego de memoria ("memorama") en el que dos jugadores compiten por descubrir la mayor cantidad de parejas de cartas posible. Las cartas se colocan boca abajo sobre un tablero y, por turnos, cada jugador selecciona dos de ellas con la intención de encontrar una coincidencia. El juego debe llevar un control preciso de los turnos, del puntaje de cada participante y del estado de cada carta, además de comunicar con claridad el resultado final de la partida.

A continuación se detallan los elementos que la interfaz debe exhibir de forma permanente durante el desarrollo del juego:

Elemento visible	Descripción
Nombre de los jugadores	Identificación clara de ambos participantes en pantalla durante toda la partida.
Turno actual	Indicador visual (color, texto o resaltado) que señale a qué jugador le corresponde jugar.
Aciertos por jugador	Contador individual que se actualice automáticamente cada vez que se descubre una pareja.
Estado del tablero	Representación gráfica de las 36 cartas y su condición (boca abajo, descubierta o emparejada).
Mensaje de ganador	Aviso final que compare los aciertos y anuncie al ganador o, en su defecto, el empate.

1. Diseño Inicial

Antes de escribir código, el equipo debe elaborar una planificación del sistema que incluya, como mínimo, un diagrama de clases (UML simplificado) y una breve descripción textual de la responsabilidad de cada clase. Este diseño debe identificar explícitamente los siguientes componentes:

- **Una (1) clase abstracta:** que represente el concepto general de "carta" y concentre el comportamiento común a todas las cartas del juego.
- **Dos (2) interfaces:** que definan los contratos de comportamiento relacionados con el control de la partida (por ejemplo, la lógica del juego y la gestión de turnos).
- **Aplicación de polimorfismo en al menos dos (2) puntos del diseño:** de modo que distintos tipos de objetos puedan tratarse de manera uniforme a través de una referencia común (clase abstracta o interfaz).
- **Clases hijas:** que hereden de la clase abstracta para representar variaciones concretas de carta dentro del juego.
- **Manejo de excepciones (try-catch):** en los puntos del programa donde puedan producirse errores de ejecución (por ejemplo, validación de datos de entrada, acceso a colecciones o carga de imágenes), evitando que el programa finalice de forma abrupta ante un evento inesperado.

Se recomienda documentar brevemente, junto al diagrama, el porqué de cada decisión de diseño (por ejemplo, qué método se declaró abstracto y por qué, o qué comportamiento se decidió aislar en una interfaz).

2. Implementación

2.1 Uso de Clase Abstracta

El sistema debe incluir una clase abstracta que represente el concepto general de una carta del juego. Esta clase debe declarar —e implementar cuando corresponda— los comportamientos comunes que todas las cartas compartirán, entre ellos:

- Mostrar la carta (revelar su imagen o valor al jugador).

- Ocultar la carta (regresarla a su estado boca abajo).
- Indicar si la carta se encuentra actualmente descubierta o no.

A partir de esta clase abstracta se deben crear las clases hijas necesarias para representar las cartas específicas del juego (por ejemplo, una carta estándar y una variante con comportamiento adicional), reutilizando la lógica común definida en la superclase y especializando únicamente aquello que sea distinto.

2.2 Uso de Interfaces(Investigar)

El sistema debe incluir al menos dos interfaces que definan las acciones principales que rigen la partida. Estas interfaces deben especificar comportamientos como:

- Iniciar el juego (preparar el tablero y las condiciones iniciales de la partida).
- Controlar los turnos (determinar y alternar qué jugador debe jugar en cada momento).
- Verificar si dos cartas seleccionadas forman una pareja.
- Finalizar la partida (detectar la condición de término y desencadenar el cierre del juego).

Una clase principal (o controladora) del juego deberá implementar estas interfaces, concentrando así la lógica de control de la partida y desacoplándola de la representación visual (GUI) y del modelo de datos (cartas y jugadores).

2.3 Aplicación del Polimorfismo

El sistema debe permitir tratar distintos tipos de cartas de manera uniforme, como si pertenecieran a un mismo tipo general. Es decir, aunque existan diferentes diseños o variantes de carta, todas deben responder correctamente a las mismas acciones definidas en la clase abstracta (mostrar, ocultar, indicar estado), de modo que el tablero pueda manipular una colección homogénea de cartas sin necesidad de conocer el tipo concreto de cada una.

2.4 Manejo de Excepciones (Try–Catch)

El programa debe incorporar bloques try–catch en los puntos críticos donde exista riesgo de error en tiempo de ejecución, por ejemplo: validación de los nombres de los jugadores, acceso a posiciones del tablero, carga o lectura de las imágenes de las cartas, y cualquier operación que dependa de datos externos o de la interacción del usuario. Ante un error, el sistema debe informar al usuario de forma clara (por ejemplo mediante un cuadro de diálogo) y mantener la aplicación en un estado estable, sin cerrarse de forma inesperada.

3. Interfaz Gráfica de Usuario (GUI)

Se recomienda desarrollar la interfaz gráfica utilizando las librerías estándar de Java (Swing), dada la facilidad que ofrecen para construir pantallas interactivas. La aplicación debe estar organizada, como mínimo, en dos pantallas:

3.1 Pantalla de Inicio

Esta pantalla debe permitir configurar la partida antes de comenzar el juego. Debe incluir:

- Un campo de texto para ingresar el nombre del Jugador 1.
- Un campo de texto para ingresar el nombre del Jugador 2.
- Un botón para iniciar el juego, que dé paso al tablero una vez validados los datos.

La partida no debe poder iniciarse si alguno de los dos nombres se encuentra vacío; en ese caso, el sistema debe mostrar un mensaje de validación indicando al usuario qué dato falta completar.

3.2 Tablero de Juego

El tablero debe presentarse como una cuadrícula de 6×6 (36 posiciones en total), organizada a partir de una colección de 12 imágenes distintas, cada una repetida dos veces para formar las parejas, y distribuidas de manera aleatoria en cada nueva partida. La pantalla del tablero debe incluir:

- Indicador visual del turno actual.
- Puntaje (cantidad de aciertos) de ambos jugadores, visible en todo momento.
- Las 36 cartas presentadas boca abajo al iniciar la partida.
- Actualización automática de la interfaz cada vez que se descubre, empareja u oculta una carta.

Se recomienda representar cada carta mediante un botón o componente gráfico interactivo, de forma que el jugador pueda seleccionarla directamente con el mouse.

4. Flujo del Juego

La lógica de la partida debe seguir la siguiente secuencia en cada turno:

1. El jugador en turno selecciona una primera carta, la cual se revela en pantalla.
2. El jugador selecciona una segunda carta, la cual también se revela en pantalla.
3. El sistema verifica automáticamente si ambas cartas forman una pareja.

En función del resultado de esa verificación, el sistema debe comportarse de la siguiente manera:

Si hay coincidencia:

- Ambas cartas permanecen visibles de forma permanente.
- Se suma un acierto al marcador del jugador en turno.
- El jugador conserva el turno y puede continuar jugando.

Si no hay coincidencia:

- Ambas cartas vuelven a ocultarse (boca abajo) tras una breve pausa que permita al jugador memorizar su posición.
- El turno pasa automáticamente al jugador contrario.

La partida finaliza en el momento en que todas las parejas del tablero han sido descubiertas.

5. Estructura de Datos del Jugador

Debe existir una estructura (clase) que represente a cada jugador, con al menos los siguientes atributos:

- Nombre del jugador.
- Cantidad de aciertos (parejas encontradas) durante la partida.

El sistema debe actualizar el puntaje de cada jugador en tiempo real, reflejando en la interfaz gráfica cualquier cambio inmediatamente después de ocurrido.

6. Finalización del Juego

Al concluir la partida —es decir, cuando ya no queden cartas boca abajo— el sistema debe:

- Comparar la cantidad de aciertos obtenidos por cada jugador.
- Mostrar un mensaje claro que indique cuál de los dos jugadores resultó ganador, mencionando su nombre y su puntaje final.
- En caso de empate en la cantidad de aciertos, notificarlo correctamente mediante un mensaje específico para esa condición, en lugar de declarar un ganador arbitrario.

7. Requisito Adicional — Equipos de 4 Integrantes

Los equipos conformados por cuatro integrantes deberán incorporar, además de todo lo anterior, un temporizador individual para cada jugador que registre el tiempo que tarda en completar el juego en caso de resultar ganador. Este cronómetro debe activarse únicamente mientras transcurre el turno del jugador correspondiente (deteniéndose o pausándose cuando el turno pase al oponente) y debe mostrarse de forma visible durante toda su participación. Al finalizar la partida, el tiempo total del jugador ganador debe mostrarse junto con el mensaje de victoria.